Flujo Detallado del Sistema de Permisos

Voy a explicar detalladamente cómo funciona el flujo del sistema de permisos propuesto, incluyendo ejemplos con datos reales para cada tabla.

1. Estructura General

El sistema de permisos se divide en dos categorías principales:

1. **Permisos del Sistema (SystemPermission)**: Controlan funciones administrativas y de gestión general
2. **Permisos de Procesos (ProcessPermission)**: Controlan acciones específicas dentro de los flujos de trabajo de la aplicación

2. Tablas y Ejemplos de Datos

SystemPermission

Esta tabla almacena los permisos generales del sistema

.

idPermission	permissionName	description	category	createdAt	updatedAt
1	USER_VIEW	Ver información de usuarios	User	2023-05-01	null
2	USER_CREATE	Crear nuevos usuarios	User	2023-05-01	null
3	USER_EDIT	Editar información de usuarios	User	2023-05-01	null
4	USER_PASSWORD_RESET	Restablecer contraseñas de usuarios	User	2023-05-01	null
5	USER_DELETE	Eliminar usuarios	User	2023-05-01	null
6	ROLE_VIEW	Ver roles	Role	2023-05-01	null

7	ROLE_CREATE	Crear nuevos roles	Role	2023-05-01	null
8	ROLE_EDIT	Editar roles existentes	Role	2023-05-01	null
9	ROLE_DELETE	Eliminar roles	Role	2023-05-01	null
10	PERMISSION_ASSIGN	Asignar permisos a roles	Permission	2023-05-01	null

ProcessPermission

Esta tabla almacena los permisos específicos para cada proceso y estado.

idProcessPermission	processId	stateId	permissionType	description	createdAt	updatedAt
1	1	null	VIEW	Ver procesos disciplinarios	2023-05-01	null
2	1	null	CREATE	Crear procesos disciplinarios	2023-05-01	null
3	1	null	EDIT	Editar procesos disciplinarios	2023-05-01	null
4	1	null	DELETE	Eliminar procesos disciplinarios	2023-05-01	null
5	1	2	STATE	Gestionar estado de resolución	2023-05-01	null
6	1	3	STATE	Gestionar estado de notificación	2023-05-01	null
7	2	null	VIEW	Ver reconocimientos	2023-05-01	null
8	2	null	CREATE	Crear reconocimientos	2023-05-01	null
9	2	null	EDIT	Editar reconocimientos	2023-05-01	null
10	2	null	DELETE	Eliminar	2023-05-	null

idProcessPermission	processId	stateId	permissionType	description	createdAt	updatedAt
				reconocimientos	01	

En este ejemplo:

- processId=1 corresponde a "Proceso Disciplinario"
- processId=2 corresponde a "Reconocimiento"
- stateId=2 corresponde al estado "Resolución" del proceso disciplinario
- stateId=3 corresponde al estado "Notificación" del proceso disciplinario

Un valor `null` en stateId significa que el permiso aplica a todo el proceso en general, no a un estado específico.

RoleSystemPermission

Esta tabla relaciona roles con permisos del sistema.

idRoleSystemPermission	roleId	systemPermissionId	createdAt	updatedAt
1	1	1	2023-05-01	null
2	1	2	2023-05-01	null
3	1	3	2023-05-01	null
4	1	4	2023-05-01	null
5	1	5	2023-05-01	null
6	1	6	2023-05-01	null
7	1	7	2023-05-01	null
8	1	8	2023-05-01	null
9	1	9	2023-05-01	null
10	1	10	2023-05-01	null
11	2	1	2023-05-01	null
12	2	3	2023-05-01	null
13	3	1	2023-05-01	null

En este ejemplo:

- roleId=1 es "Administrador" y tiene todos los permisos
- roleId=2 es "Supervisor" y puede ver usuarios (1) y editarlos (3)
- roleId=3 es "Usuario" y solo puede ver usuarios (1)

RoleProcessPermission

Esta tabla relaciona roles con permisos de procesos.

idRoleProcessPermission	roleId	processPermissionId	createdAt	updatedAt
1	1	1	2023-05-01	null
2	1	2	2023-05-01	null
3	1	3	2023-05-01	null
4	1	4	2023-05-01	null
5	1	5	2023-05-01	null
6	1	6	2023-05-01	null
7	1	7	2023-05-01	null
8	1	8	2023-05-01	null
9	1	9	2023-05-01	null
10	1	10	2023-05-01	null
11	2	1	2023-05-01	null
12	2	2	2023-05-01	null
13	2	3	2023-05-01	null
14	2	5	2023-05-01	null
15	2	7	2023-05-01	null
16	2	8	2023-05-01	null
17	3	1	2023-05-01	null
18	3	7	2023-05-01	null

En este ejemplo:

- roleId=1 (Administrador) tiene todos los permisos de procesos
- roleId=2 (Supervisor) puede ver, crear y editar procesos disciplinarios, gestionar resoluciones, y ver y crear reconocimientos
- roleId=3 (Usuario) solo puede ver procesos disciplinarios y reconocimientos

3. Flujo de Funcionamiento

3.1. Inicialización del Sistema

Cuando el sistema se inicia por primera vez, se deben crear:

1. **Roles básicos**:
    - Administrador (puede hacer todo)
    - Supervisor (puede gestionar procesos pero no usuarios)
    - Usuario (acceso limitado a visualización)
2. **Permisos del sistema**:
    - Todos los permisos listados en la tabla SystemPermission
3. **Permisos de procesos**:
    - Todos los permisos listados en la tabla ProcessPermission
4. **Asignaciones iniciales**:

- Asignar todos los permisos al rol Administrador
- Asignar permisos limitados a Supervisor y Usuario

3.2. Flujo de Login y Carga de Permisos

1. **Usuario inicia sesión**:

```
2. POST /api/Login
   { "email": "admin@example.com", "password": "Admin123!" }
```

3. **Sistema autentica y genera token JWT**:
   - Verifica credenciales contra la tabla User
   - Incluye roleId en el token JWT
   - Devuelve token y menús asignados al rol
4. **Frontend almacena información**:
   - Guarda token en localStorage: `authToken`
   - Guarda ID de usuario en localStorage: `userId`
   - Guarda menús asignados en localStorage: `menuRoles`
5. **Frontend carga permisos del usuario**:

```
javascript
// Después del login exitoso
dispatch(fetchUserPermissions());  // Acción Redux
```

6. **Backend procesa la solicitud de permisos**:
   - Identifica el usuario por su token
   - Obtiene su rol
   - Consulta permisos asignados a ese rol
   - Devuelve la lista completa de permisos
7. **Frontend almacena permisos**:

```
javascript
// Reducer de Redux
builder.addCase(fetchUserPermissions.fulfilled, (state, action) =>
{
  state.userPermissions = action.payload;
  localStorage.setItem("userPermissions",
JSON.stringify(action.payload));
});
```

3.3. Verificación de Permisos en el Frontend

Verificación en Rutas

```jsx
// Proteger ruta que requiere permiso de sistema
<Route path='/users/create' element={
  <PermissionLayout systemPermission="USER_CREATE">
    <UserCreatePage />
  </PermissionLayout>
} />

// Proteger ruta que requiere permiso de proceso
<Route path='/disciplinary-process/new' element={
  <PermissionLayout
    processId={1}
    processPermissionType="CREATE"
  >
    <DisciplinaryProcessPage />
  </PermissionLayout>
} />
```

Verificación en Componentes UI

```jsx
// Mostrar botón solo si tiene permiso
<ConditionalRender
  processId={1}
  processPermissionType="CREATE"
>
  <button onClick={handleCreateNew}>
    Nuevo Proceso
  </button>
</ConditionalRender>
```

Usando Hooks Personalizados

```jsx
// En un componente
const { hasPermission, loading } = useSystemPermission('USER_CREATE');
```

```
// Renderizado condicional
{hasPermission && (
  <button onClick={handleCreateUser}>Crear Usuario</button>
)}
```

3.4. Verificación de Permisos en el Backend

1. **Usuario realiza una acción protegida**:
2. POST /api/DisciplinaryProcess
   { ... datos del proceso ... }

3. **Controlador verifica permisos antes de procesar**:

```
csharp
[HttpPost]
public async Task<IActionResult>
CreateDisciplinaryProcess([FromForm] string process, [FromForm]
List<IFormFile> files)
{
    ResponseApi response = new ResponseApi();
    try
    {
        // Obtener ID del usuario actual
        string userId = _userService.GetCurrentUserId();
        if (string.IsNullOrEmpty(userId))
        {
            throw new Exception("Usuario no logueado");
        }

        // Verificar permiso para crear proceso disciplinario
        bool hasPermission = await
_permissionService.UserHasProcessPermission(
            int.Parse(userId),
            1,  // processId para Proceso Disciplinario
            null,
            "CREATE"
```

```
            );

            if (!hasPermission)
            {
                return Forbid(); // 403 Forbidden
            }

            // Continuar con la creación del proceso...
        }
        catch (Exception ex)
        {
            response.Success = false;
            response.Message = ex.Message;
            return BadRequest(response);
        }
    }
```

3.5. Administración de Permisos

1. **Administrador accede a la interfaz de gestión de permisos**:
   - Navega a `/permissions`
   - El componente `PermissionLayout` verifica si tiene permiso `PERMISSION_ASSIGN`
2. **Interfaz muestra roles y permisos actuales**:
   - Lista de roles (Administrador, Supervisor, Usuario, etc.)
   - Permisos del sistema agrupados por categoría
   - Permisos de procesos agrupados por proceso
3. **Administrador selecciona un rol**:
   - Sistema carga los permisos asignados a ese rol
   - Muestra checkboxes marcados para los permisos asignados
4. **Administrador modifica permisos**:
   - Marca/desmarca checkboxes para asignar/revocar permisos
   - Cada acción envía una solicitud al servidor:

```javascript
// Asignar permiso
await axiosInstance.post('/Permission/assign/system', {
  roleId: selectedRole.idRole,
  permissionId: permissionId
```

```
});

// Revocar permiso
await axiosInstance.post('/Permission/revoke/system', {
  roleId: selectedRole.idRole,
  permissionId: permissionId
});
```

5. **Backend procesa los cambios**:
   - Verifica que el usuario tiene permiso `PERMISSION_ASSIGN`
   - Actualiza la tabla `RoleSystemPermission` o `RoleProcessPermission`
   - Devuelve respuesta de éxito o error

4. Escenarios de Uso Comunes

4.1. Usuario Estándar (roleId=3)

Juan es un empleado normal que usa el sistema para:

- Ver procesos disciplinarios (permiso: ProcessId=1, Type=VIEW)
- Ver reconocimientos (permiso: ProcessId=2, Type=VIEW)

Cuando Juan inicia sesión:

1. Recibe su token JWT con roleId=3
2. El frontend carga sus permisos
3. Solo ve los menús y botones para funciones a las que tiene acceso
4. Si intenta acceder directamente a una URL restringida, es redirigido

4.2. Supervisor (roleId=2)

María es supervisora y puede:

- Ver, crear y editar procesos disciplinarios
- Gestionar resoluciones de procesos
- Ver y crear reconocimientos
- Ver y editar algunos usuarios

Cuando María accede al módulo de procesos disciplinarios:

1. Puede ver la lista completa
2. Ve el botón "Nuevo Proceso" porque tiene permiso CREATE
3. Puede entrar al detalle de cada proceso

4. En procesos en estado "Resolución", ve el formulario para crear resoluciones
5. No puede eliminar procesos (no tiene permiso DELETE)

4.3. Administrador (roleId=1)

Carlos es administrador y tiene acceso total:

- Todos los permisos del sistema
- Todos los permisos de procesos
- Acceso a la interfaz de administración de permisos

Carlos puede:

1. Crear nuevos usuarios y asignarles roles
2. Crear nuevos roles con permisos personalizados
3. Modificar permisos existentes
4. Realizar cualquier acción en procesos disciplinarios y reconocimientos

5. Ventajas de este Sistema

1. **Flexibilidad**:
    - Permite permisos granulares por proceso y estado
    - Fácil de extender para nuevos procesos
2. **Seguridad**:
    - Verificación de permisos tanto en frontend como backend
    - Jerarquía clara de roles y permisos
3. **Mantenibilidad**:
    - Estructura clara y bien definida
    - Fácil de entender y modificar
4. **Usabilidad**:
    - Interfaz intuitiva para administrar permisos
    - Los usuarios solo ven lo que pueden utilizar

Este sistema de permisos proporciona una base sólida para controlar el acceso a las diferentes funcionalidades de la aplicación, adaptándose a las necesidades específicas de cada rol y proceso.